Social Media Impact on Students' Life

Complete Machine Learning Project Documentation

Dataset: Social_media_impact_on_life.csv
Programming Language: Python
Environment: Jupyter Notebook
Libraries: Pandas, NumPy, Matplotlib, Seaborn, Scikit-learn

---

1. Project Introduction

The purpose of this project is to analyze how social media affects students' lives using Machine Learning.

We use students' information, such as daily social media usage, sleep duration, stress score, academic performance, and other features to predict the overall impact of social media.

Our target variable is `Overall_Impact`, which has three classes:

1. Beneficial
2. Negative
3. Neutral

We train three Machine Learning models and compare their performance to select the best model.

2. Dataset Information

Our dataset contains:

- Total Rows: 4500
- Total Columns: 16
- Total Records: 4500 students
- Target Column: Overall_Impact

Dataset Columns

Column	Description
Student_ID	Unique student identification
Age	Student's age
Gender	Student's gender
Academic_Level	Academic level
Primary_Platform	Main social media platform
Daily_Usage_Hours	Daily social media usage
Weekend_Extra_Hours	Extra weekend usage
Device_Type	Device used
Sleep_Duration_Hours	Sleeping hours
Sleep_Quality_Score	Sleep quality score
Late_Night_Usage	Whether social media is used late at night
Social_Comparison_Frequency	Frequency of social comparison
Perceived_Stress_Score	Perceived stress score
Mental_Health_Index	Mental health index
Academic_Performance_GPA	Academic GPA
Overall_Impact	Target variable

TASK 1: Import Libraries

Code

```
None
import pandas as pd
import numpy as np
```

```
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.model_selection import StratifiedKFold
from sklearn.model_selection import cross_val_score

from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer

from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.ensemble import GradientBoostingClassifier

from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score
from sklearn.metrics import classification_report
from sklearn.metrics import ConfusionMatrixDisplay
```

Purpose

Importing libraries means bringing ready-made Python tools into our program.

Why did we use these libraries?

- **Pandas:** To read and manage the dataset.
- **NumPy:** To perform numerical operations.
- **Matplotlib:** To create graphs.
- **Seaborn:** To create statistical visualizations.
- **Scikit-learn:** To preprocess data, train models, and evaluate performance.

Important concept

A library is a collection of ready-made functions that helps us perform different tasks without writing everything from scratch.

TASK 2: Load Dataset

Code

```
None
data_df = pd.read_csv("Social_media_impact_on_life.csv")

data_df.head()
```

Purpose

To load our CSV dataset into Python.

Explanation

- `pd.read_csv()` reads a CSV file.
- `data_df` is the variable that stores our dataset.
- `head()` displays the first five rows.

Expected Output

The first five records of the dataset are displayed in tabular form.

Why?

Before applying Machine Learning, we must load and understand the data.

TASK 3: Explore Dataset

We inspect the dataset to understand its structure and quality.

3.1 Check Dataset Shape

```
None
data_df.shape
```

Output

```
None
(4500, 16)
```

Explanation

- 4500 represents the number of rows.
- 16 represents the number of columns.

Purpose: To know how much data we have.

3.2 Check Column Names

```
None
data_df.columns
```

Purpose: To see all available features and identify the target column.

3.3 Check Dataset Information

```
None
data_df.info()
```

Purpose: To check:

- Column names
- Data types
- Number of non-null values
- Memory usage

Important concept: Data Types

- `int`: Integer numbers.
- `float`: Decimal numbers.
- `object`: Text or categorical values.

- `bool`: True or False values.

3.4 Statistical Description

```
None
data_df.describe()
```

Purpose: To understand numerical data using:

- Count
- Mean
- Standard deviation
- Minimum
- Maximum
- Quartiles

3.5 Check Missing Values

```
None
data_df.isnull().sum()
```

Output

```
None
Perceived_Stress_Score       46
Academic_Performance_GPA     85
```

Total missing values:

```
None
131
```

Purpose: To identify incomplete records.

3.6 Check Duplicate Records

```
None
data_df.duplicated().sum()
```

Output

```
None
0
```

Explanation: There are no duplicate rows.

3.7 Check Target Distribution

```
None
data_df['Overall_Impact'].value_counts()
```

Output

```
None
Beneficial    3681
Neutral        654
Negative       165
```

Purpose: To understand the number of records in each target class.

Important Concept: Class Imbalance

Class imbalance occurs when one class contains significantly more records than other classes.

Our dataset has many Beneficial records but fewer Negative records.

Therefore, we should not judge our model using accuracy alone. Macro F1-score is also important.

TASK 4: Exploratory Data Analysis (EDA)

EDA means Exploratory Data Analysis.

It is the process of understanding data using statistics and visualizations before training Machine Learning models.

Purpose

- Understand relationships between features.
- Find patterns in the dataset.
- Identify unusual values.
- Understand the distribution of data.

Example 1: Target Distribution

```
None
plt.figure(figsize=(8, 5))

sns.countplot(
    data=data_df,
    x='Overall_Impact'
)

plt.title("Distribution of Overall Social Media Impact")
plt.xlabel("Overall Impact")
plt.ylabel("Number of Students")

plt.show()
```

Reason: To compare the number of students in Beneficial, Negative, and Neutral categories.

Example 2: Daily Usage Distribution

```
None
plt.figure(figsize=(8, 5))

sns.histplot(
    data_df['Daily_Usage_Hours'],
    kde=True
)

plt.title("Daily Social Media Usage")
plt.xlabel("Daily Usage Hours")
plt.ylabel("Number of Students")

plt.show()
```

Reason: To understand how many hours students spend on social media.

Example 3: Sleep Duration

```
None
plt.figure(figsize=(8, 5))

sns.histplot(
    data_df['Sleep_Duration_Hours'],
    kde=True
)

plt.title("Sleep Duration Distribution")
plt.xlabel("Sleep Duration Hours")
plt.show()
```

Reason: To understand students' sleeping-hour distribution.

Example 4: Correlation Heatmap

```
None
plt.figure(figsize=(12, 8))

sns.heatmap(
    data_df.select_dtypes(include=np.number).corr(),
    annot=True,
    cmap='coolwarm'
)

plt.title("Feature Correlation Heatmap")
plt.show()
```

Reason: To understand the correlation between numerical features.

Important Concept: Correlation

Correlation describes how two numerical variables are related.

- Positive correlation: Both variables tend to increase together.
- Negative correlation: One tends to increase when the other decreases.

- Near-zero correlation: Little or no linear relationship.

Correlation does not automatically prove causation.

TASK 5: Data Cleaning and Preprocessing

Data preprocessing means preparing raw data before giving it to a Machine Learning model.

5.1 Handle Missing Values

We found:

- Perceived_Stress_Score: 46 missing values.
- Academic_Performance_GPA: 85 missing values.

We filled them using the median.

Code

```
None
data_df['Perceived_Stress_Score'] =
data_df['Perceived_Stress_Score'].fillna(
    data_df['Perceived_Stress_Score'].median()
)

data_df['Academic_Performance_GPA'] =
data_df['Academic_Performance_GPA'].fillna(
    data_df['Academic_Performance_GPA'].median()
)
```

Verify Missing Values

```
None
data_df.isnull().sum()
```

Output

Both columns now contain zero missing values.

What is Median?

Median is the middle value when numbers are arranged in ascending order.

Why Median?

Median is less affected by unusually high or low values than the mean.

5.2 Remove Student_ID

Code

```
None
data_df = data_df.drop(columns=['Student_ID'])
```

Purpose

Student_ID is only an identification number. It does not provide meaningful information about social media impact.

Therefore, we remove it from the dataset.

5.3 Separate Features and Target

Code

```
None
X = data_df.drop(columns=['Overall_Impact'])

y = data_df['Overall_Impact']
```

Explanation

- `X`: Input features used for prediction.
- `y`: Output or target that we want to predict.

Important Concept

In supervised Machine Learning, the model learns from input features and their known target labels.

5.4 Separate Numerical and Categorical Columns

Code

```
None
numeric_columns = X.select_dtypes(
    include=np.number
).columns.tolist()

categorical_columns = X.select_dtypes(
    include=['object', 'bool', 'category']
).columns.tolist()
```

Output

Numerical columns: 8

```
None
Age
Daily_Usage_Hours
Weekend_Extra_Hours
Sleep_Duration_Hours
Sleep_Quality_Score
Perceived_Stress_Score
Mental_Health_Index
Academic_Performance_GPA
```

Categorical columns: 6

```
None
Gender
Academic_Level
Primary_Platform
Device_Type
Late_Night_Usage
Social_Comparison_Frequency
```

Why?

Machine Learning algorithms generally require numerical inputs.

We must handle numerical and categorical data differently.

5.5 Train-Test Split

Code

```
None
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)
```

Output

```
None
Training Data: (3600, 14)
Testing Data: (900, 14)
```

Explanation

We divided the dataset into:

- 80% Training Data
- 20% Testing Data

Important Parameters

test_size=0.20

20% of the data is reserved for testing.

random_state=42

Keeps the random split reproducible.

stratify=y

Maintains approximately the same target-class proportions in training and testing data.

Important Concept: Training and Testing

- Training data teaches the model.
- Testing data evaluates the model on records it did not use for fitting.

5.6 Create Preprocessing Pipelines

Code

```
None
numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])

categorical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('encoder', OneHotEncoder(handle_unknown='ignore'))
])

preprocessor = ColumnTransformer(transformers=[
    ('num', numeric_transformer, numeric_columns),
    ('cat', categorical_transformer, categorical_columns)
])
```

Purpose

To automatically prepare numerical and categorical columns.

Numerical Transformer

SimpleImputer(strategy='median')

Fills missing numerical values using the median.

StandardScaler()

Scales numerical values so they have a comparable scale.

Categorical Transformer

SimpleImputer(strategy='most_frequent')

Fills missing categorical values using the most common category.

OneHotEncoder()

Converts categories into numerical columns.

What is One-Hot Encoding?

For example, suppose Gender contains:

```
None
Male
Female
```

One-hot encoding converts categories into separate numerical columns:

Male	Female
1	0
0	1

Why handle_unknown='ignore'?

It prevents errors if the test dataset contains a category that was not present during training.

5.7 Apply Preprocessing

Code

```
None
X_train_processed = preprocessor.fit_transform(X_train)

X_test_processed = preprocessor.transform(X_test)
```

Output

```
None
Training Shape: (3600, 32)
Testing Shape: (900, 32)
```

Explanation

`fit_transform()` learns preprocessing rules from training data and applies them.

`transform()` applies the already learned rules to test data.

Why?

We must not learn preprocessing information from the test dataset. This helps prevent data leakage.

TASK 6: Train Three Machine Learning Models

We selected three classification algorithms.

1. Logistic Regression
2. Random Forest
3. Gradient Boosting

6.1 Logistic Regression

Code

```
None
model_1 = LogisticRegression(
    max_iter=1000,
    class_weight='balanced',
    random_state=42
)
```

```
model_1.fit(X_train_processed, y_train)
```

Purpose

Logistic Regression is a classification algorithm used to predict categories.

Important Parameters

- `max_iter=1000`: Allows more optimization iterations.
- `class_weight='balanced'`: Gives more importance to less frequent classes.
- `random_state=42`: Makes results reproducible.

6.2 Random Forest

Code

```
None
model_2 = RandomForestClassifier(
    n_estimators=200,
    class_weight='balanced',
    random_state=42
)

model_2.fit(X_train_processed, y_train)
```

Purpose

Random Forest combines multiple decision trees to make predictions.

Important Parameter

`n_estimators=200` means the model uses 200 decision trees.

6.3 Gradient Boosting

Code

```
None
model_3 = GradientBoostingClassifier(
    n_estimators=100,
    learning_rate=0.1,
    random_state=42
)

model_3.fit(X_train_processed, y_train)
```

Purpose

Gradient Boosting builds models sequentially. Each new model attempts to improve on the errors of previous models.

Important Parameters

- `n_estimators=100`: Number of boosting stages.
- `learning_rate=0.1`: Controls how much each stage contributes.
- `random_state=42`: Supports reproducibility.

6.4 Make Predictions

Code

```
None
prediction_1 = model_1.predict(X_test_processed)

prediction_2 = model_2.predict(X_test_processed)

prediction_3 = model_3.predict(X_test_processed)
```

Purpose

Each trained model predicts the target class for the test dataset.

TASK 7: Compare Model Performance

We compare the three models using performance metrics.

Code

```
None
model_results = pd.DataFrame({
    'Model': [
        'Logistic Regression',
        'Random Forest',
        'Gradient Boosting'
    ],

    'Accuracy': [
        accuracy_score(y_test, prediction_1),
        accuracy_score(y_test, prediction_2),
        accuracy_score(y_test, prediction_3)
    ],

    'Precision': [
        precision_score(y_test, prediction_1, average='macro'),
        precision_score(y_test, prediction_2, average='macro'),
        precision_score(y_test, prediction_3, average='macro')
    ],

    'Recall': [
        recall_score(y_test, prediction_1, average='macro'),
        recall_score(y_test, prediction_2, average='macro'),
        recall_score(y_test, prediction_3, average='macro')
    ],

    'F1-Score': [
        f1_score(y_test, prediction_1, average='macro'),
        f1_score(y_test, prediction_2, average='macro'),
        f1_score(y_test, prediction_3, average='macro')
    ]
})

print(model_results)
```

Purpose

To compare the performance of all three models.

Important Evaluation Metrics

Accuracy: Percentage of all predictions that are correct.

Precision: Of the records predicted as a particular class, how many actually belong to that class?

Recall: Of the actual records in a class, how many did the model correctly identify?

F1-Score: A combined measure of precision and recall.

Macro Average: Calculates the metric for each class separately and then gives all classes equal importance.

Why Macro F1?

Our dataset is imbalanced. Beneficial has many more records than Negative.

Macro F1 helps us evaluate all three classes fairly.

TASK 8: Initial Cross-Validation

We used StratifiedKFold for cross-validation.

Code

```
None
cv = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)

cv_scores = cross_val_score(
    best_model,
    X_train_processed,
```

```
    y_train,
    cv=cv,
    scoring='f1_macro'
)

print("CV Scores:", cv_scores)
print("Average CV Score:", cv_scores.mean() * 100)
```

Purpose

Cross-validation checks model performance across multiple training and validation splits.

What is 5-Fold Cross-Validation?

The training data is divided into five parts.

The model trains five times, each time using four parts for training and one part for validation.

Important Note

This initial approach used data preprocessing fitted before the folds. Therefore, it can cause data leakage between cross-validation folds.

For a more reliable evaluation, we created complete pipelines in Task 9.

TASK 9: Proper Pipeline Cross-Validation

We created complete pipelines so that preprocessing is learned separately inside every cross-validation training fold.

9.1 Create Model Pipelines

Code

```
None
pipeline_1 = Pipeline(steps=[
```

```
    ('preprocessing', preprocessor),
    ('model', LogisticRegression(
        max_iter=1000,
        class_weight='balanced',
        random_state=42
    ))
])

pipeline_2 = Pipeline(steps=[
    ('preprocessing', preprocessor),
    ('model', RandomForestClassifier(
        n_estimators=200,
        class_weight='balanced',
        random_state=42
    ))
])

pipeline_3 = Pipeline(steps=[
    ('preprocessing', preprocessor),
    ('model', GradientBoostingClassifier(
        n_estimators=100,
        learning_rate=0.1,
        random_state=42
    ))
])
```

Purpose

A complete pipeline combines preprocessing and model training into one workflow.

This ensures that preprocessing is fitted only on the training portion of each fold.

9.2 Compare Models Using Cross-Validation

Code

```
None
model_pipelines = {
    'Logistic Regression': pipeline_1,
    'Random Forest': pipeline_2,
    'Gradient Boosting': pipeline_3
}

cv_results = {}

for model_name, model_pipeline in model_pipelines.items():

    scores = cross_val_score(
        model_pipeline,
        X_train,
        y_train,
        cv=cv,
        scoring='f1_macro',
        n_jobs=-1
    )

    cv_results[model_name] = scores

    print(model_name)
    print("Scores:", scores)
    print("Average:", scores.mean() * 100)
    print()
```

Purpose

To compare all three models using the same five-fold validation method.

9.3 Create Comparison Table

Code

```
None
final_comparison = pd.DataFrame({
```

```
    'Model': list(cv_results.keys()),

    'Average CV F1 (%)': [
        scores.mean() * 100
        for scores in cv_results.values()
    ],

    'Standard Deviation (%)': [
        scores.std() * 100
        for scores in cv_results.values()
    ]
}).sort_values(
    by='Average CV F1 (%)',
    ascending=False
).reset_index(drop=True)

print(final_comparison.to_string(index=False))
```

Purpose

To display the average Macro F1 and standard deviation for each model.

9.4 Select Best Model

Code

```
None
best_model_name = final_comparison.iloc[0]['Model']

best_pipeline = model_pipelines[best_model_name]

print("Selected Model:", best_model_name)

print(
    "Average CV Macro F1:",
    round(
```

```
        final_comparison.iloc[0]['Average CV F1 (%)'],
        2
    ),
    "%"
)
```

Your Output

```
None
Selected Model: Gradient Boosting
Average CV Macro F1: 94.37 %
```

Explanation

Gradient Boosting achieved the highest average cross-validation Macro F1-score among the three models.

Therefore, we selected it as our best model.

TASK 10: Train Final Model

Code

```
None
best_pipeline.fit(X_train, y_train)

print("Final model training completed successfully!")
```

Output

```
None
Final model training completed successfully!
```

Purpose

To train the selected Gradient Boosting pipeline using the complete training dataset.

Important Concept

The selected pipeline automatically performs preprocessing and model training.

We use the original `X_train` because preprocessing is already included in the pipeline.

TASK 11: Final Model Evaluation

Code

```
None
final_predictions = best_pipeline.predict(X_test)

final_accuracy = accuracy_score(
    y_test,
    final_predictions
)

final_f1 = f1_score(
    y_test,
    final_predictions,
    average='macro'
)

print("Final Model Performance")
print("-----------------------")

print(
    "Accuracy:",
    round(final_accuracy * 100, 2),
    "%"
)

print(
```

```
    "Macro F1-Score:",
    round(final_f1 * 100, 2),
    "%"
)
```

Your Output

```
None
Final Model Performance
-----------------------
Accuracy: 97.33 %
Macro F1-Score: 93.53 %
```

Explanation

Accuracy = 97.33%

The model correctly classified 876 out of 900 test records.

Macro F1 = 93.53%

The model achieved strong balanced performance across the three target classes.

TASK 12: Classification Report

Code

```
None
print("Classification Report")
print("----------------------")

print(classification_report(
    y_test,
    final_predictions,
    zero_division=0
```

```
))
```

Your Output

```
None
              precision    recall  f1-score   support

Beneficial       0.99      0.99      0.99       736
Negative         0.91      0.91      0.91        33
Neutral          0.90      0.92      0.91       131

accuracy                             0.97       900

macro avg        0.93      0.94      0.94       900
weighted avg     0.97      0.97      0.97       900
```

Explanation

Beneficial

- Precision: 99%
- Recall: 99%
- F1-score: 99%
- Support: 736

Negative

- Precision: 91%
- Recall: 91%
- F1-score: 91%
- Support: 33

Neutral

- Precision: 90%
- Recall: 92%
- F1-score: 91%
- Support: 131

What is Support?

Support is the actual number of records belonging to a particular class in the test dataset.

Macro Average

The average performance of all three classes, giving equal importance to each class.

Weighted Average

The average performance of all three classes, taking the number of records in each class into account.

TASK 13: Confusion Matrix

Code

```
None
from sklearn.metrics import ConfusionMatrixDisplay
import matplotlib.pyplot as plt

ConfusionMatrixDisplay.from_predictions(
    y_test,
    final_predictions,
    cmap='Blues'
)

plt.title("Gradient Boosting - Confusion Matrix")

plt.show()
```

Your Confusion Matrix Values

Actual Class	Predicted Beneficial	Predicted Negative	Predicted Neutral
Beneficial	726	0	10
Negative	0	30	3
Neutral	8	3	120

Explanation

Beneficial

Out of 736 actual Beneficial records:

- 726 were correctly predicted.
- 10 were predicted as Neutral.

Negative

Out of 33 actual Negative records:

- 30 were correctly predicted.
- 3 were predicted as Neutral.

Neutral

Out of 131 actual Neutral records:

- 120 were correctly predicted.
- 8 were predicted as Beneficial.
- 3 were predicted as Negative.

Total Correct Predictions

None

```
726 + 30 + 120 = 876
```

Total Incorrect Predictions

None

```
10 + 3 + 8 + 3 = 24
```

Total Test Records

None

```
876 + 24 = 900
```

The confusion matrix confirms the final accuracy of approximately 97.33%.

FINAL PROJECT CONCLUSION

In this project, we analyzed the impact of social media on students' lives using Machine Learning.

We performed data exploration, data cleaning, missing-value handling, feature selection, encoding, scaling, and train-test splitting.

We trained and compared three Machine Learning models:

1. Logistic Regression
2. Random Forest
3. Gradient Boosting

After comparing the models using five-fold cross-validation, Gradient Boosting was selected as the best model.

Final Results

Evaluation	Result
Selected Model	Gradient Boosting
Average CV Macro F1	94.37%
Final Test Accuracy	97.33%
Final Test Macro F1	93.53%
Correct Predictions	876
Incorrect Predictions	24

The final model performed well in classifying social media's overall impact into Beneficial, Negative, and Neutral categories.

Project Status: Completed Successfully.

IMPORTANT CONCEPTS FOR VIVA

Question	Simple Answer

What is Machine Learning?	A method that allows computers to learn patterns from data.
What is Classification?	Predicting a category or class.
What is a Dataset?	A collection of data arranged in rows and columns.
What are Features?	Input columns used to make predictions.
What is a Target?	The output we want to predict.
What is Preprocessing?	Preparing data before model training.
What is Missing Value?	A value that is not available in the dataset.
What is Median?	The middle value of ordered numbers.
What is Encoding?	Converting categorical data into numerical form.
What is Scaling?	Adjusting numerical features to a comparable scale.
What is Train-Test Split?	Dividing data into training and testing portions.
What is Cross-Validation?	Evaluating a model through multiple data splits.
What is Logistic Regression?	A classification algorithm.
What is Random Forest?	A model that combines multiple decision trees.
What is Gradient Boosting?	A boosting algorithm that builds models sequentially to improve predictions.
What is Accuracy?	The proportion of correct predictions.
What is Precision?	How many predicted positive records are actually positive.
What is Recall?	How many actual positive records are correctly identified.
What is F1-Score?	A combined measure of precision and recall.
What is Confusion Matrix?	A table showing correct and incorrect predictions.
Why did we select Gradient Boosting?	It achieved the highest average cross-validation Macro F1 among our three models.

IMPORTANT TECHNICAL NOTE

The initial model comparison used the test dataset before the final cross-validation-based selection. Therefore, the final test score should be treated as an evaluation on previously viewed test data, not as a completely untouched independent final assessment.

For a strictly independent final evaluation, a fresh holdout dataset should be reserved before model selection.

Also, high performance does not by itself establish that social media causes a particular impact. The model learns patterns and associations in the supplied dataset.